Список заданий

Общая постановка задачи:

Необходимо согласно приведенному ниже списку заданий разработать новый программный компонент в рамках компонентной архитектуры (ACOM). Создаваемый компонент должен расширять базовую математическую функциональность системы, включающую операции сложения и вычитания, и реализовывать новый интерфейс с методами согласно выбранной задаче.

Для большинства вычислительных задач интерфейс должен предоставлять перегруженные методы (функции) для работы с различными типами данных (int, long, float, double, long double).

Пример для задачи 25: Необходимо реализовать вычисление значения возведенного основания в степень. При решении данной задачи интерфейс должен содержать следующие методы:

```
typedef struct IEcoCalculatorZ {

    /* IEcoUnknown */
    …

    /* IEcoCalculatorZ */
    double (*pow)(/* in */ struct IEcoCalculatorZ* me, /* in */ double x, /* in */ double
y); // с плавающей точкой двойной точности
    float (*powf)(/* in */ struct IEcoCalculatorZ* me, /* in */ float x, /* in */ float
y); // с плавающей точкой
    long double (*powl)(/* in */ struct IEcoCalculatorZ* me, /* in */ long double x, /*
in */ long double y); // с плавающей точкой повышенной точности

} IEcoCalculatorZ, *IEcoCalculatorZPtr;
```

Дополнительно:

Для задач которые имеют несколько способов решения (алгоритмов) необходимо создать дополнительно компоненты реализующие эти алгоритмы. В клиенте «Калькулятор» предусмотреть выбор того или иного алгоритма для загрузки соответствующего компонента. Пример для ***задачи 1***: Необходимо реализовать вычисление квадратного корня. При решении данной задачи существует несколько алгоритмов:

1) Метод половинного деления
2) Метод Ньютона

Соответственно для каждого алгоритма должна быть своя реализация компонента.

```
/* EcoCalculatorMethod1 CID = {FC8DE947-EDF2-43CC-8C71-1BF22FFEDB9E} */
#ifndef __CID_EcoCalculatorMethod1
static const UGUID CID_EcoCalculatorMethod1 = {0x01, 0x10, 0xFC, 0x8D, 0xE9, 0x47, 0xED,
0xF2, 0x43, 0xCC, 0x8C, 0x71, 0x1B, 0xF2, 0x2F, 0xFE, 0xDB, 0x9E};
#endif /* __CID_EcoCalculatorMethod1 */

/* EcoCalculatorMethod2 CID = {0F88C3E6-D005-4667-83BC-06B045FC071F} */
#ifndef __CID_EcoCalculatorMethod2
```

```
static const UGUID CID_EcoCalculatorMethod2 = {0x01, 0x10, 0x0F, 0x88, 0xC3, 0xE6, 0xD0,
0x05, 0x46, 0x67, 0x83, 0xBC, 0x06, 0xB0, 0x45, 0xFC, 0x07, 0x1F};
#endif /* __CID_EcoCalculatorMethod2 */
```

Для задач повышенной и высшей сложности:

Подразумевается создание типов данных и интерфейсов для работы с комплексными числами, векторами и матрицами. Пример для ***задачи 69*** («Создание матрицы с заданными диагональными элементами / функция diag»). При решении данной задачи необходимо спроектировать интерфейсы, входные и выходные параметры с учетом полного запрета на функции динамического выделения памяти из стандартной библиотеки (libc):

```
typedef struct IEcoLinearAlgebra {

    /* IEcoUnknown */
    ...

    /* IEcoLinearAlgebra */
    IEcoVector* (*new_Vector)(/* in */ struct IEcoLinearAlgebra* me, /* in */ ...);
    IEcoMatrix* (*new_Matrix)(/* in */ struct IEcoLinearAlgebra* me, /* in */ ...);
    IEcoMatrix* (*set_Diag)(/* in */ struct IEcoLinearAlgebra* me, /* in */ IEcoVector*
vector, /* in */ ...);
    IEcoVector* (*get_Diag)(/* in */ struct IEcoLinearAlgebra* me, /* in */ IEcoMatrix*
matrix, /* in */ ...);
    ...

} IEcoLinearAlgebra, *IEcoLinearAlgebraPtr;
```

Примечание:

Запрещается использование стандартных библиотечных функций (библиотеки libc), а также сторонних библиотек и фреймворков.

Источники информации и базы знаний:

Для поиска математического описания алгоритмов, формул разложения в ряды и псевдокода студентам разрешается использовать любые открытые базы знаний: специализированные интернет-ресурсы, профильную онлайн-литературу, электронные учебники, а также статьи в **Википедии** (свободной энциклопедии). Изучение алгоритмов по статьям в Википедии и математическим справочникам является рекомендуемым этапом проектирования, так как позволяет детально разобраться во внутренней логике функций перед началом написания кода.

К примеру для задачи 25: https://en.wikipedia.org/wiki/Exponentiation#Power_functions

Обязательное требование к выполнению и тестированию:

Работоспособность и корректность разработанного компонента должна быть полностью подтверждена с **Unit-теста**. Тестовый клиент должен представлять собой консольное приложение.

Студент имеет право выполнять задание как **полностью самостоятельно**, так и **с использованием инструментов искусственного интеллекта (ИИ-ассистентов)** для генерации начального кода и прототипов алгоритмов. Однако нейросети склонны к галлюцинациям, синтаксическим ошибкам и неявному использованию запрещенных функций libc.

Независимо от выбранного способа разработки (вручную или с помощью ИИ), **студент обязан лично провести сквозную верификацию итогового кода**.

Раздел 1. Задачи элементарной математики (Интерфейс `IEcoMathElementary`)

Все задачи данного раздела требуют реализации без использования стандартных макросов и функций `libc`. Оценка зависит от точности и количества реализованных алгоритмов.

1. **Вычисление квадратного корня числа (функция `sqrt`)**
   - *4-5 баллов:* Реализация одним методом (например, метод половинного деления).
   - *6-7 баллов:* Создание двух отдельных компонентов с разными алгоритмами (Метод половинного деления и Метод Ньютона) с возможностью динамического выбора клиентом.
2. **Вычисление кубического корня числа (функция `cbrt`)** - *Категория: Базовая (4-5 баллов).*
3. **Вычислениеского корня n-ной степени (функция `nthrt`)** - *Категория: Базовая (4-5 баллов).*
4. **Вычисление значения экспоненты (функция `exp`)** - через разложение в ряд Маклорена. *Категория: Базовая (4-5 баллов).*
5. **Вычисление значения $x \cdot 2^{exp}$ (функция `ldexp`)** - *Категория: Базовая (4-5 баллов).*
6. **Вычисление мантиссы и степени числа с плавающей запятой (функция `frexp`)**
   - *4-5 баллов:* Базовый алгоритм.
   - *6-7 баллов:* Реализация через прямой побитовый разбор структуры floating-point числа (стандарт IEEE 754) вручную.
7. **Вычисление натурального (по основанию e) логарифма числа (функция `log`)** - *Категория: Базовая (4-5 баллов).*
8. **Вычисление десятичного (по основанию 10) логарифма числа (функция `log10`)** - *Категория: Базовая (4-5 баллов).*
9. **Вычисление двоичного (по основанию 2) логарифма числа (функция `log2`)** - *Категория: Базовая (4-5 баллов).*
10. **Вычисление косинуса числа (функция `cos`)** - *Категория: Базовая (4-5 баллов).*
11. **Вычисление арккосинуса числа в радианах (функция `acos`)** - *Категория: Базовая (4-5 баллов).*
12. **Вычисление гиперболического косинуса числа (функция `cosh`)** - *Категория: Базовая (4-5 баллов).*
13. **Вычисление гиперболического арккосинуса числа (функция `acosh`)** - *Категория: Базовая (4-5 баллов).*

14. **Вычисление синуса числа (функция `sin`)** - *Категория: Базовая (4-5 баллов).*
15. **Вычисление арксинуса числа (функция `asin`)** - *Категория: Базовая (4-5 баллов).*
16. **Вычисление гиперболического синуса числа (функция `sinh`)** - *Категория: Базовая (4-5 баллов).*
17. **Вычисление гиперболического арксинуса числа (функция `asinh`)** - *Категория: Базовая (4-5 баллов).*
18. **Вычисление тангенса числа (функция `tan`)** - *Категория: Базовая (4-5 баллов).*
19. **Вычисление арктангенса числа (функция `atan`)** - *Категория: Базовая (4-5 баллов).*
20. **Вычисление арктангенса для заданных координат (функция `atan2`)** - *Категория: Базовая (4-5 баллов).*
21. **Вычисление гиперболического тангенса числа (функция `tanh`)** - *Категория: Базовая (4–5 баллов).*
22. **Вычисление гиперболического арктангенса числа (функция `atanh`)** - *Категория: Базовая (4-5 баллов).*
23. **Вычисление абсолютного значения числа (функция `abs` / `fabs`)** - *Категория: Минимальная (4 балла).*
24. **Округление числа до ближайшего большего и меньшего целого (функции `ceil` и `floor`)** - *Категория: Минимальная (4 балла).*
25. **Вычисление значения возведенного основания в степень (функция `pow`)** - *Категория: Базовая (4-5 баллов).*
26. **Вычисление наибольшего и наименьшего из двух значений (функции `max` и `min`)** - *Категория: Минимальная (4 балла).*
27. **Вычисление остатка от деления с плавающей точкой (функция `fmod`)** - *Категория: Базовая (4-5 баллов).*
28. **Вычисление целочисленной и дробной части числа (функция `modf`)** - *Категория: Базовая (4-5 баллов).*
29. **Вычисление длины гипотенузы прямоугольного треугольника (функция `hypot`)** - *Категория: Базовая (4-5 баллов).*
30. **Преобразование угла из градусов в радианы и наоборот (функции `deg2rad` и `rad2deg`)** - *Категория: Минимальная (4 балла).*

Раздел 2. Продвинутые спецфункции и Численный анализ (Интерфейс `IEcoAdvancedMath`)

31. **Вычисление функции Бесселя первого рода порядка n (функции `j0`, `j1`, `jn`)**
    - *5-6 баллов:* Реализация только для фиксированных порядков `j0` и `j1`.
    - *8-9 баллов:* Повышенная сложность - реализация функции `jn` для произвольного порядка через рекуррентные соотношения.
32. **Вычисление функции Бесселя второго рода порядка n (функции `y0`, `y1`, `yn`)**
    - *5-6 баллов:* Реализация только для `y0` и `y1`.
    - *8-9 баллов:* Повышенная сложность - реализация общей функции `yn`.
33. **Вычисление натурального логарифма от гамма-функции числа (функция `lgamma`)** - *Категория: Средняя (5-7 баллов).*

34. **Вычисление гамма-функции от числа (функция `tgamma`)** - *Категория: Средняя (5-7 баллов).*
35. **Численное адаптивное интегрирование (функция `integral`)** - реализация метода Гаусса-Кронрода или метода Симпсона. *Категория: Повышенная (7-8 баллов).*
36. **Решение систем обыкновенных дифференциальных уравнений (ОДУ)** - метод Рунге-Кутты 4-го порядка (аналог `ode45` в MATLAB). *Категория: Высшая сложность (9-10 баллов).*
37. **Кубическая сплайн-интерполяция (функция `spline`)** - построение гладкой кривой через набор точек с ручной реализацией метода прогонки для трехдиагональной матрицы. *Категория: Высшая сложность (9-10 баллов).*

Раздел 3. Генераторы псевдослучайных чисел (Интерфейс `IEcoPseudoRandom`)

38. **Алгоритм регистра с линейной обратной связью LFSR-k в конфигурации Фибоначчи** - *Категория: Базовая (4-5 баллов).*
39. **Алгоритм регистра с линейной обратной связью LFSR-k в конфигурации Галуа** - *Категория: Базовая (4-5 баллов).*
40. **Алгоритм генерации псевдослучайных чисел Xorshift** - *Категория: Базовая (4-5 баллов).*
    - *Критерий на 6-7 баллов за задачи 38-40:* Реализация всех трех алгоритмов в виде независимых компонентов с возможностью динамического переключения генератора в клиенте.

Раздел 4. Алгебра полиномов и Рациональные дроби (Интерфейс `IEcoPolynomialMath`)

41. **Приближение числа в виде рациональной дроби (функция `rat`)** - *Категория: Средняя (5-6 баллов).*
42. **Вычисление коэффициентов полинома по его корням (функция `poly`)** - *Категория: Средняя (5-6 баллов).*
43. **Нахождение корней полинома (функция `roots`)** - *Категория: Повышенная (7-8 баллов).*
44. **Нахождение коэффициентов полинома методом наименьших квадратов (функция `polyfit`)** - *Категория: Повышенная (7-8 баллов).*
45. **Разложение рациональной дроби на простейшие (функция `residue`)** - *Категория: Средняя (5-7 баллов).*
46. **Свертка и умножение полиномов (функция `conv`)** - *Категория: Средняя (5-6 баллов).*
47. **Деконволюция по методу наименьших квадратов и полиномиальное деление (функция `deconv`)** - *Категория: Средняя (5-7 баллов).*

Раздел 5. Задачи поиска и упорядочивания элементов (Интерфейс `IEcoSortAndSearch`)

Все алгоритмы сортировки (кроме распределительных) должны строго соответствовать прототипу функции `qsort`, работать через универсальные указатели `void*` и принимать пользовательскую функцию сравнения элементов.

- **Подраздел 5.1. Базовые алгоритмы сортировки и поиска (Оценка: 4-5 баллов)**
  48. Алгоритм - Сортировка пузырьком / Bubble sort
  49. Алгоритм - Сортировка перемешиванием / Cocktail sort
  50. Алгоритм - Сортировка вставками / Insertion sort
  51. Алгоритм - Гномья сортировка / Gnome sort
  52. Алгоритм - Сортировка выбором / Selection sort
  53. Алгоритм - Придурковатая сортировка / Stooge sort
  54. Алгоритм - Линейный поиск в массиве (`linear search`)
- **Подраздел 5.2. Продвинутые алгоритмы сортировки и поиска (Оценка: 5-6 баллов)**
  55. Алгоритм - Сортировка слиянием / Merge sort
  56. Алгоритм - Сортировка с помощью двоичного дерева / Tree sort
  57. Алгоритм - Сортировка расчёской / Comb sort
  58. Алгоритм - Сортировка Шелла / Shell sort
  59. Алгоритм - Быстрая сортировка / Quicksort
  60. Алгоритм - Пирамидальная сортировка / Heapsort (Кучей)
  61. Алгоритм - Двоичный (бинарный) поиск в массиве (`bsearch`)
- **Подраздел 5.3. Высокопроизводительные и специализированные алгоритмы (Оценка: 6–8 баллов)**
  62. Алгоритм - Сортировка Timsort (гибридный алгоритм)
  63. Алгоритм - Плавная сортировка / Smoothsort (на куче Эдсгера Дейкстры)
  64. Алгоритм - Битонная сортировка / Bitonic sort (параллельная сортировка)
  65. Алгоритм - Блочная сортировка / Bucket sort (распределительная)
  66. Алгоритм - Поразрядная сортировка / Radix sort (распределительная)
  67. Алгоритм - Сортировка подсчётом / Counting sort (распределительная)
  68. Алгоритм - Внешняя многопутевая сортировка слиянием (сортировка файлов, размер которых превышает объем ОЗУ).

Раздел 6. Обработка сигналов и Линейная алгебра (Интерфейсы `IEcoLinearAlgebra` / `IEcoDSP`)

Требуют предварительного проектирования собственных интерфейсов для векторов (`IEcoVector`) и матриц (`IEcoMatrix`), а также структур комплексных чисел без использования `libc`.

69. **Создание матрицы с заданными диагональными элементами или выделение диагонали (функция `diag`)** - *Категория: Базовая (4-5 баллов).*
70. **Быстрое преобразование Фурье (функция `fft`)** - перевод дискретного сигнала из временной области в частотную. *Категория: Повышенная (7-8 баллов).*

71. **Обратное быстрое преобразование Фурье (функция `ifft`)** - *Категория: Повышенная (7-8 баллов).*
72. **Реализация цифрового фильтра 1-D (функция `filter`)** - одномерная фильтрация сигналов и временных рядов. *Категория: Средняя (5–6 баллов).*
73. **Реализация цифрового фильтра 2-D (функция `filter2`)** - двумерная фильтрация матриц (изображений) с использованием ядер свертки. *Категория: Повышенная (7-8 баллов).*
74. **Матричные разложения общего назначения и СЛАУ (LU / QR / SVD)**
    - *6-8 баллов:* Реализация LU или QR разложения матрицы для решения СЛАУ методом Гаусса.
    - *9-10 баллов:* Высшая сложность - Сингулярное разложение матрицы (`svd`) для задач компрессии данных.

Раздел 7. Математические компоненты ИИ (Интерфейс `IEcoArtificialIntelligence`)

Специализированный раздел высшей сложности.

75. **Комплекс функций активации нейронных сетей и их градиентов**
    - *5-6 баллов:* Реализация только расчета прямых значений функций `softmax`, `sigmoid`, `ReLU`, `Leaky ReLU` над структурами векторов и матриц.
    - *9-10 баллов:* Высшая сложность — обязательное добавление методов вычисления градиентов (производных) для каждой из функций активации, необходимых для алгоритма обратного распространения ошибки (Backpropagation).
76. **Многомерный градиентный спуск (Gradient Descent)** - реализация оптимизатора весов модели, принимающего указатель на кастомную функцию потерь (например, MSE), шаг обучения и вектор весов. *Категория: Высшая сложность (9-10 баллов).*
77. **Построение K-мерного дерева (K-d tree)** - реализация структуры пространственного дерева и алгоритма поиска ближайшего соседа (k-NN) для задач классификации в машинном обучении. *Категория: Высшая сложность (9-10 баллов).*

Отчетность:

- Предоставить исходный код реализованного компонента и его сборку.

- Реализацию клиента использующего компонент(-ы), в виде калькулятора или UnitTest’a с несколькими примерами.

- Документ (формат word) содержащий описание компонента и его функциональность. А также примеры и сравнительные таблицы результатов вычислений при использовании стандартных библиотек в Си #include “math.h” или MATLAB с их аналогами, реализованными в виде компонентов (сравнение точности до N знаков после запятой). Для задач повышенной и высшей сложности (разделы 5.3, 6, 7) дополнительно предоставить анализ производительности (замеры времени

выполнения или тактов процессора средствами ОС/клиента) в сравнении со стандартными аналогами.

Критерии оценивания:

0 - 3 баллов - за невыполненную работу, наличие плагиата или грубые ошибки (код не компилируется, падает, используются запрещенные функции libc).

4 - 5 баллов - за выполненную базовую работу (задачи с меткой «Базовая» или подраздел 5.1) согласно заданию с недочетами, без предоставления отчета.

6 - 7 баллов - за выполненную работу базового или повышенного уровня (подразделы 5.2, 5.3, разделы 3, 4) без ошибок, с предоставлением отчета и сравнительных таблиц точности.

8 баллов - за выполненную без ошибок работу повышенной сложности (раздел 2, раздел 6) с предоставлением полного отчета, включая сравнительный анализ точности и выводы.

9 - 10 баллов - за безошибочную реализацию задач **категории «Высшая сложность»** (численные методы Рунге-Кутты, сплайны, SVD-разложение или математические компоненты ИИ с расчетом градиентов из раздела 7) с проектированием собственных интерфейсов данных, предоставлением полного отчета, анализом точности и замерами производительности

Примечание (Дополнительные баллы):

Студент имеет возможность повысить итоговую оценку за лабораторную работу (но **не более чем на 2–3 балла суммарно** в рамках 10-балльной системы), применив продвинутые технологии автоматизации разработки или реализовав дополнительные архитектурные подходы.

Дополнительные баллы начисляются по двум независимым направлениям:

Направление 1. Инжиниринг и оптимизация системных промтов (Prompt Engineering в ACOM) — до 2 баллов

Баллы начисляются, если студент в случае использования ИИ не просто вел хаотичный диалог в чате, а спроектировал, протестировал и задокументировал **эффективный системный промт (System Prompt / Инструкцию)**, заставляющий ИИ гарантированно и корректно генерировать код в рамках жестких ограничений экосистемы ACOM.

1. **Состав промта:** Промт должен четко задавать для ИИ роль системного программиста, правила интерфейсов ACOM (передача указателя `me`, разделение `in`/`out`), категорический запрет на подключение и вызовы любых функций `libc` (включая неявные вроде `memcpy`), а также требования по автоматическому внедрению проверок на `NULL` и крайние математические случаи.

2. **Требования к отчету:** В отчет добавляется подраздел *«Инжиниринг промтов и автоматизация разработки»*, где фиксируется финальный текст промта, история его итеративной оптимизации (какие ошибки ИИ делал сначала и как промт их исправил) и пример эталонного сгенерированного кода.

Направление 2. Архитектурное усложнение (OOA vs SOA) - до 2 баллов

Баллы начисляются, если компонент для задач из **Разделов 3, 5, 6 или 7** реализован двумя принципиально разными архитектурными способами одновременно, с демонстрацией обоих подходов в Unit-тесте:

3. **Сервисно-ориентированным (Stateless Service):** Компонент не хранит состояние; все данные для вычислений передаются в методы как параметры извне.
4. **Объектно-ориентированным (Stateful Object):** Компонент инкапсулирует состояние (данные, массивы, коэффициенты или структуры) внутри себя, а его методы управляют этим внутренним состоянием.
   В клиенте «Калькулятор» должна быть предусмотрена демонстрация работы обоих архитектурных подходов, а в отчете - приведено сравнение их удобства использования и накладных расходов на память.

1. Группа «Сортировки и Поиск» (Раздел 5)

- **Сервисный подход (SOA):** Классический `qsort`. Компонент-сервис предоставляет метод `Sort`, в который клиент каждый раз передает указатель на внешний массив данных, его размер и функцию сравнения. Компонент отсортировал и «забыл».
- **Объектный подход (OOA):** Создается полноценный объект «Контейнер/Коллекция» (например, `IEcoArray`). Сначала вызываются методы `Insert`, объект накапливает элементы во внутренней структуре данных, а затем вызывается метод `Sort()`, который упорядочивает внутреннее состояние этого конкретного объекта.

2. Группа «Генераторы псевдослучайных чисел LFSR» (Разд. 3, задачи 38-40)

- **Сервисный подход (SOA):** Метод принимает текущее состояние регистра (seed/маску) извне в качестве аргумента, делает один сдвиг и возвращает новое состояние. Сам компонент ничего не помнит.
- **Объектный подход (OOA):** Объект инициализируется один раз методом `Init(seed)`. Текущее состояние и полином обратной связи хранятся в приватных полях компонента. Клиент просто вызывает метод `Next()`, а объект сам внутри себя обновляет состояние и выдает новое число.

3. Группа «Обработка сигналов и Линейная алгебра» (Раздел 6, задачи 69-74)

- **Сервисный подход (SOA):** Функция `filter` или `filter2` оформлена как чистый сервис обработки. Клиент передает указатель на исходную матрицу, указатель на матрицу-фильтр (ядро) и указатель на матрицу-результат.
- **Объектный подход (OOA):** Создается объект `IEcoDigitalFilter`. При создании в него «зашиваются» коэффициенты фильтра или ядро свертки. Затем этот объект

можно использовать многократно, пропуская через него разные сигналы (метод `Apply(signal)`), при этом объект может хранить внутреннюю историю (внутреннее состояние задержек для IIR-фильтров).

4. Группа «Математические компоненты ИИ» (Раздел 7, задачи 75-77)

- **Сервисный подход (SOA):** Функции активации (`ReLU`, `Sigmoid`). Принимают вектор, возвращают преобразованный вектор. Состояние не требуется.
- **Объектный подход (OOA):** Компонент «Нейронный Слой» (`IEcoNeuralLayer`). Он хранит внутри себя матрицы весов, смещений (biases), а также кэширует промежуточные выходы (активации) после прямого прохода, чтобы использовать их при вызове метода обратного прохода (`Backward`) для расчета градиентов.